

Dining Hall and Meal Plan Management System

Shawn Will, 11811009; Riley Nielsen, 11794884

Professor Parteek Kumar

April 27th, 2025

CPTS 451 - Introduction to Database Systems

Table of Contents

Table of Contents.....	1
Abstract.....	2
Introduction.....	3
Overview:.....	3
Problem Statement:.....	3
Objectives/Scope:.....	3
Potential Impact:.....	3
Functional and Non-Functional Requirements.....	5
Functional Requirements:.....	5
User Stories:.....	6
Non-Functional Requirements:.....	6
Database Design.....	8
ER Diagram:.....	8
ER to Relational Tables:.....	8
Table Structure:.....	9
Relationships:.....	10
Schema of Tables:.....	12
SQL Implementation.....	13
DDL Scripts:.....	13
DML Scripts:.....	13
Feature Implementations.....	15
Query Design.....	17
User Interface Snapshots.....	19
Limitations & Future Scope.....	22
Conclusion.....	23

Abstract

In an increasingly digital world, university campuses must modernize their dining hall services to meet the evolving expectations of students. This project proposes the design and implementation of a dining hall and meal plan tracking system that enhances the dining experience for both students and administrators.

For students, the system provides a seamless interface to monitor their meal plan balance, review transaction history, explore menus, and submit feedback on dining services. By offering real-time budget tracking and access to past purchases, students can make informed dining decisions and manage their meal plans more effectively. Additionally, the ability to submit feedback ensures that students have a voice in improving campus dining services.

For administrators, the system offers robust tools for managing menu updates, tracking popular items, and analyzing student preferences. Dining hall management can optimize meal offerings, reduce food waste, and make informed decisions about menu planning and inventory. The feedback system further enables administrators to identify areas for improvement, ensuring a high standard of service and satisfaction.

By integrating a database-driven system into dining hall operations, this project aims to create a more efficient, transparent, and student-centric meal plan management system. The platform not only enhances the student dining experience but also supports administrators to improve campus dining services.

Introduction

Overview:

This project's goal is to create a Dining Hall and Meal Plan system that allows students to use and manage their meal plans, as well as allow an admin to access menus and data to keep the dining halls running smoothly. This system would be used at any school and could be used by an admin to set up menus with food items and their individual nutritional information.

Problem Statement:

The problem we are trying to solve here is that some schools do not have proper meal plan applications, and with a system like this they could easily implement a meal plan system that both students and admins could easily use to make their lives run a bit smoother. Many meal plans still run entirely in-person, so students do not know what each day is going to bring. Alternatively, the meal plan and menus are often listed separately, making it harder for both users and administration to view and update in the system. This project attempts to remedy this problem.

Objectives/Scope:

Some of our main goals are as follows; we want to create a database that stores student data (names, student IDs, meal plan type, etc.), a system where a new student can register to the system and pick a meal plan, a system for a student to view the menu(s) available, a system for the student to view their current balance as well as check their past purchase history, a system to quantitatively keep track of what menu items have been bought and apply it to a usage report, a system for students to give feedback on food and service, a way for an admin to manage student data as well as the menus themselves, a way for an admin to check the usage report, and finally a way for admins to view the feedback given.

Potential Impact:

The implementation of this dining hall and meal plan management system has the potential to significantly improve the efficiency and accessibility of campus dining services.

Many schools struggle with mismanaged dining operations, leading to student dissatisfaction, budget inefficiencies, and food waste. By providing a solution that streamlines meal tracking, menu management, and feedback collection, this project empowers both students and administrators to engage with the dining system more effectively.

For students, the platform ensures they have clear visibility into their meal plan usage, allowing for better budgeting and informed dining choices. The ability to review past purchases and access up-to-date menu information enhances convenience and reduces uncertainty about available dining options. Additionally, a structured feedback system gives students a voice in shaping their dining experience, promoting a more responsive and student-friendly environment.

For administrators, the system offers valuable insights into dining trends, popular menu items, and student preferences. This can help optimize inventory management, reduce waste, and ensure that resources are allocated efficiently. By simplifying menu updates and improving communication between students and dining staff, the system enhances operational transparency and responsiveness.

While existing solutions for dining hall management do exist, there is always room for improvement. Our goal is to identify gaps in current systems and develop an intuitive, scalable platform that can be realistically adopted by schools. By addressing pain points such as inefficient meal tracking, limited student input, and lack of cohesiveness among meal plans and menus, this project aims to create a dining hall management system that is both practical and transformative.

Functional and Non-Functional Requirements

Functional Requirements:

- Students need to be able to register to the system
 - Students will register a password for the system and be given a student number, after registration, they will need to use their student number and password to access the system. Can be changed by an admin if required.
- Students need to be able to select meal plans
 - After a student is registered, they can select a meal plan for their account. After it is selected, the meal plan can only be upgraded, but never downgraded.
- There needs to be a food menu that gives descriptions of each menu item
 - The food menu will store the name of each food item, its price, description, and nutritional info. Can be updated by an admin.
- Students need to be able to access the meal plan tracker to show purchase history.
 - The meal plan tracker will have a list of past purchases over the last school year. Every school year, the purchases list will be purged for all students. This list will show the date of purchase, total order price, and which items were bought
- Students also need access to the meal plan tracker to check their remaining balance
 - This balance will be updated with every purchase.
- A student should be able to access a feedback system for dining hall services
 - The student will be able to open the feedback section of the app and choose what they would like to give feedback on.
- The feedback system will allow students to rate things like their meal, the service they got, cleanliness of the facility, and how helpful this app is
 - From what the student chose to report on, the feedback will be stored in a table with the report type and their response.
- There should also be a usage reports system that tracks the overall usage of the meal plan service
 - This system will track every menu item purchased and keep track of how many times an item was purchased.

- Admins should be able to manage menus, meal plans, and usage reports
 - Admins will be shown an alternate screen that allows them to choose what they want to manage. For the three systems listed, each will be built in a way that allows an admin to update it.
- Admins will also have access to the student register list to be able to update any student info as needed
 - If a student needs to update their student number, their password, or unregister from the meal service, an admin can access the table and update accordingly.

User Stories:

- As a student, I want to register a password so that I can be given a student number and log into the meal plan service.
- As a student, I want to select a meal plan so I can get access to funds to pay for meals.
- As a student, I want to be able to see each menu item and its nutritional info so I can plan what I want to eat.
- As a student, I want to be able to see my past purchase history so that I can see how I've used my meal plan.
- As a student, I want to be able to access a meal plan tracker so that I can check my remaining balance.
- As a student, I want to be able to give ratings on food and service so any complaints can be properly addressed.
- As an administrator, I want to be able to see feedback on services, so we can address any concerns or health and safety issues.
- As an administrator, I want to be able to track every item purchased so that I can manage usage reports.
- As an administrator, I want to be able to manage the menus, meal plans, and usage reports so that I can keep the meal plan service functioning properly.

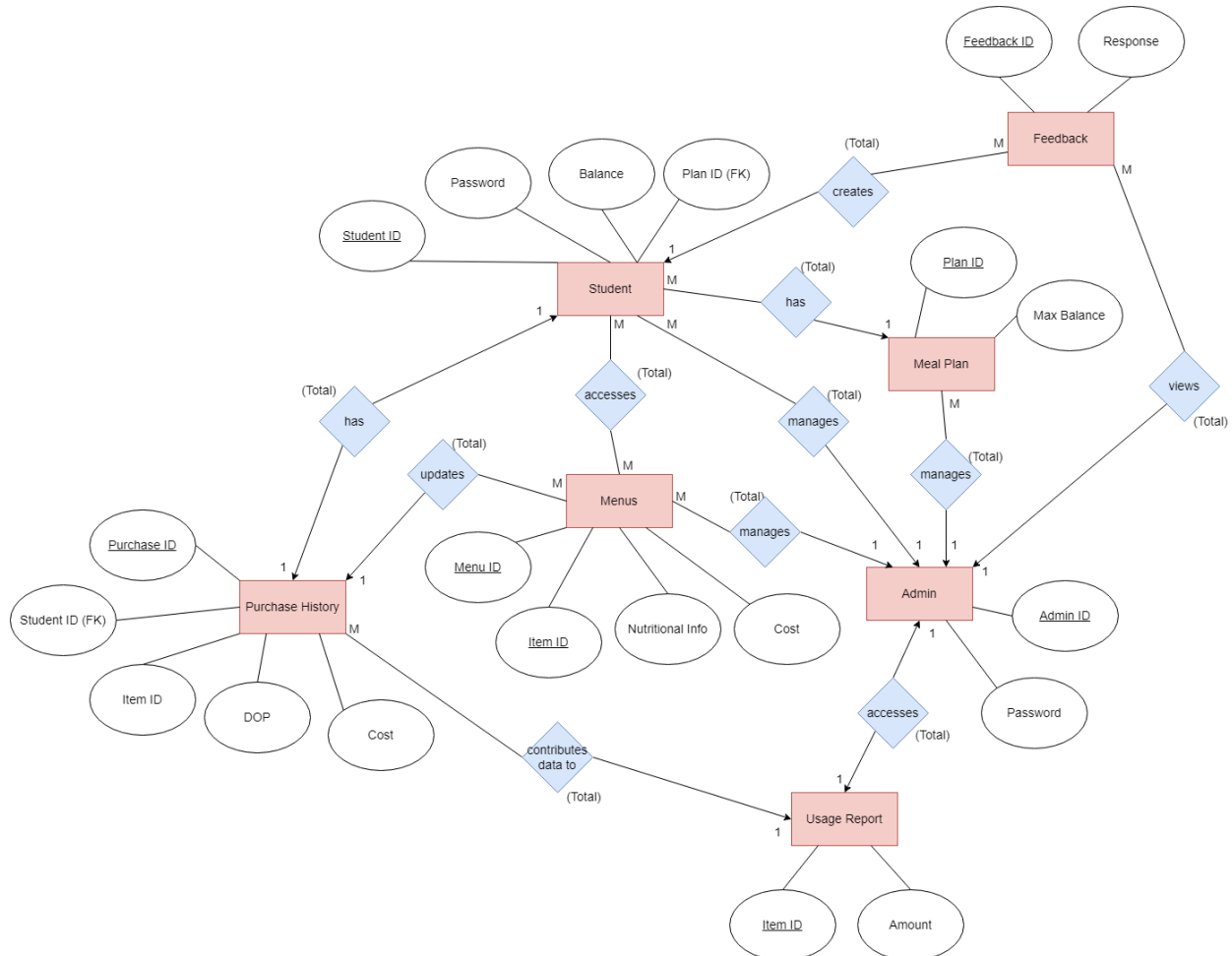
Non-Functional Requirements:

- User/admin authentication and secure logins
 - No passwords in plaintext

- Fast and lightweight to prevent lag, stutters, long loading times, etc
 - Prefer server-side processing over client-side processing, minimize packet size
- Scalable to handle large volumes of students, meal plans, and orders
 - Orders could number in the millions, so a way to handle these is a must
- Easy to use user and admin interfaces
 - Not cluttered, clean, etc
- Minimal downtime during regular operating hours
 - Ensure customer satisfaction
- Compliant with relevant online safety/privacy laws
 - Comply with FERPA
- Easy to add/update meal plan and food info
 - Cannot remove money from student account but can add money, and new food items will need to be added
- Keep an accurate record of transactions
 - Ensure database integrity
- Accessible for students with disabilities
 - Follow all applicable ADA laws, add alt text, good UI scaling, light/dark mode, etc.
- Possible support for multiple languages
 - Primarily English, but add other languages upon request using Google Translate
- Moderation for inappropriate feedback/reviews
 - Users can report reviews, and reviews are automatically screened for inappropriate content

Database Design

ER Diagram:



ER to Relational Tables:

Converting our ER diagram into tables is relatively simple because there are no complex data attributes and the relationships for each entity are clearly described. Other than the clear foreign key attributes, the tables will communicate more through our code than directly referencing each other. This is because we want it to be able to handle lots of data, and fluffing tables with unneeded references will only slow down our system. It makes more sense to us to handle these relationships through code rather than through table structure.

Table Structure:

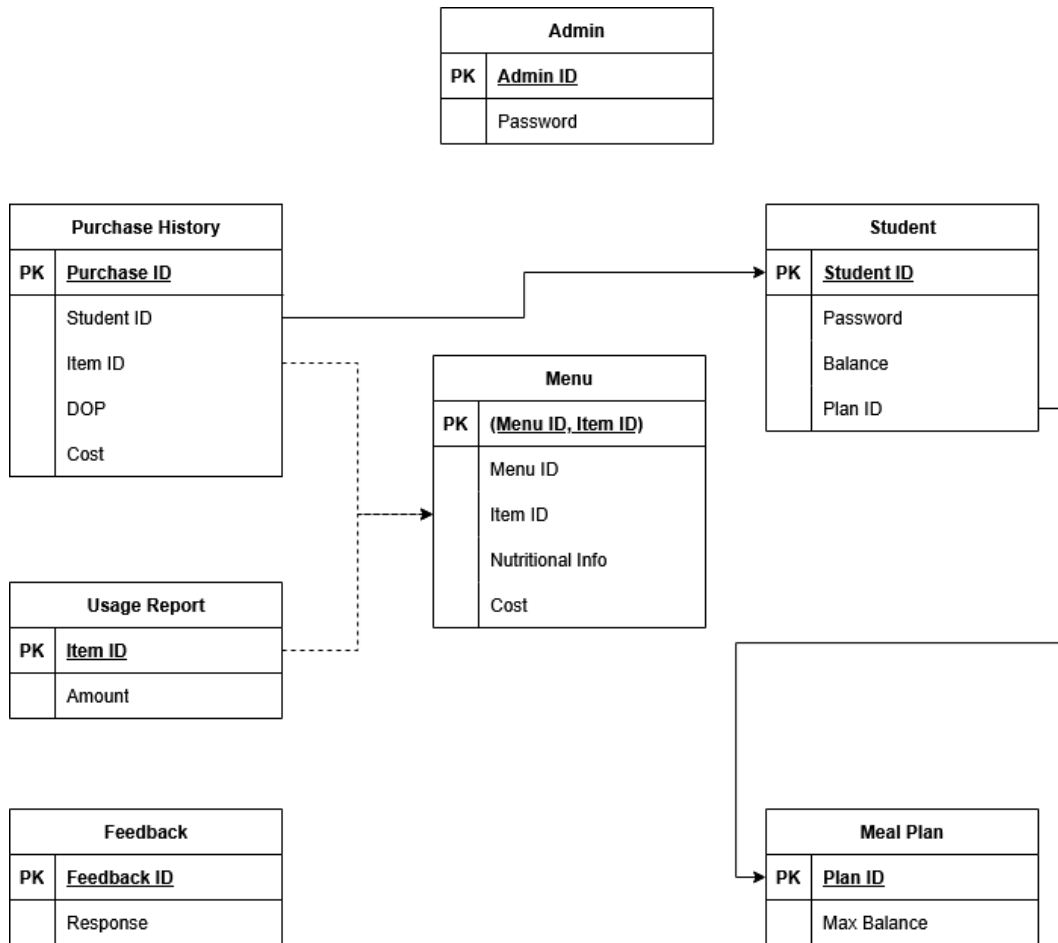
- Student
 - Student ID, Number(8), Primary Key
 - Password, VarChar(30), Not NULL
 - Balance, Decimal(8,2), Not NULL
 - Plan ID, Number(2), Foreign Key referencing Meal Plan(Plan ID)
- Meal Plan
 - Plan ID, Number(2), Primary Key
 - Max Balance, Number(6), Not NULL
- Menus
 - Menu ID, Number(3)
 - Item ID, VarChar(20)
 - Nutritional Info, Text, Not NULL
 - Cost, Decimal(6,2), Not NULL
 - Primary Key (Menu ID, Item ID)
- Purchase History
 - Purchase ID, Number(8), Primary Key
 - Student ID, Number(8), Foreign Key referencing Student(Student ID), on DELETE CASCADE
 - Item ID, VarChar(20), Not NULL
 - DOP, Date, Not NULL
 - Cost, Decimal(6,2), Not NULL
- Feedback
 - Feedback ID, Number(8), Primary Key
 - Response, Text, Not NULL
- Admin
 - Admin ID, Number (8), Primary Key
 - Password, VarChar(30), Not NULL
- Usage Report
 - Item ID, Number(4), Not NULL
 - Amount, Number(8)

Relationships:

- Student to Meal Plan
 - Each student upon registering gets to pick one meal plan. This meal plan is then connected to the student's account, and can only be changed by an admin. This meal plan also sets the max balance that the student can have, and will be references when money is added to the student balance (which is also done by the admin)
- Student to Feedback
 - Students can each create multiple feedback responses into the system. This text is just plainly stored, and cannot be edited or deleted by the student who created it. Students can make as many feedback entries as they like.
- Student to Menus
 - Students can view all current menus displayed, which includes the items in each menu, the nutritional info for each item, and the cost for each item.
- Purchase History to Student and Menu
 - When a purchase is processed, Purchase History will collect the student ID of the student who made the purchase from the student table, the item ID for the item actually purchased from the Menu table, the date of purchase (DOP), and the cost from the Menu table. Note that the item ID in the History is not a foreign key, because if the item is removed from all menus, the student would still want to know the name of the item they purchased.
- Student to Purchase History
 - Students can also view their purchase history by accessing the Purchase History table using their Student ID. This will bring up all of the data of the purchases for the student.
- Usage Report to Purchase History
 - After a new purchase is logged in the purchase history, the Report will look at the item(s) purchased by their item ID and increment the counter for that item. Not the item ID in the Report is not a foreign key, because if the item is removed from all menus, the admin would still want the data for that item.

- Admin to Student
 - Admins can change any data stored in the Student table, including removing students who wish to unregister.
- Admin to Menus
 - Admins can create new menus, remove old ones, as well as create and remove items from a menu. They can update the nutritional info for an item, as well as the cost.
- Admin to Meal Plans
 - Admins can create and remove meal plans, but the maximum balance cannot be changed. Before a meal plan can be removed, all students' meal plans must be changed by the admin to a new plan so that removal can take place.
- Admin to Usage Report
 - Admins can view the usage report.
- Admin to Feedback
 - Admins can view all feedback, and remove them as they are addressed.

Schema of Tables:



SQL Implementation

DDL Scripts:

The following SQL scripts were generated using Django's *sqlmigrate* command.

```
CREATE TABLE `diningHall_feedback` (`id` bigint AUTO_INCREMENT NOT NULL  
PRIMARY KEY, `response` longtext NOT NULL);
```

```
CREATE TABLE `diningHall_mealplan` (`id` bigint AUTO_INCREMENT NOT NULL  
PRIMARY KEY, `max_balance` numeric(8, 2) NOT NULL);
```

```
CREATE TABLE `diningHall_menu` (`menu_id` integer UNSIGNED NOT NULL  
CHECK (`menu_id` >= 0), `item_id` varchar(20) NOT NULL, `info` longtext NOT NULL,  
`cost` numeric(6, 2) NOT NULL, PRIMARY KEY (`menu_id`, `item_id`));
```

```
CREATE TABLE `diningHall_report` (`item_id` varchar(20) NOT NULL PRIMARY  
KEY, `amount` integer UNSIGNED NOT NULL CHECK (`amount` >= 0));
```

```
CREATE TABLE `diningHall_student` (`id` bigint AUTO_INCREMENT NOT NULL  
PRIMARY KEY, `balance` numeric(8, 2) NOT NULL, `meal_plan_id` bigint NOT NULL,  
`user_id` integer NOT NULL UNIQUE);
```

```
CREATE TABLE `diningHall_purchasehistory` (`id` bigint AUTO_INCREMENT NOT  
NULL PRIMARY KEY, `item_id` varchar(20) NOT NULL, `dop` date NOT NULL, `cost`  
numeric(6, 2) NOT NULL, `student_id` bigint NOT NULL);
```

```
ALTER TABLE `diningHall_student` ADD CONSTRAINT  
`diningHall_student_meal_plan_id_225c38d4_fk_diningHal` FOREIGN KEY (`meal_plan_id`)  
REFERENCES `diningHall_mealplan` (`id`);
```

```
ALTER TABLE `diningHall_student` ADD CONSTRAINT  
`diningHall_student_user_id_ab6b1a0b_fk_auth_user_id` FOREIGN KEY (`user_id`)  
REFERENCES `auth_user` (`id`);
```

```
ALTER TABLE `diningHall_purchasehistory` ADD CONSTRAINT  
`diningHall_purchaseh_student_id_fdca9c43_fk_diningHal` FOREIGN KEY (`student_id`)  
REFERENCES `diningHall_student` (`id`);
```

DML Scripts:

```
-- Inserting Meal Plans
```

```
INSERT INTO diningHall_mealplan (id, max_balance) VALUES (1, '100.00');
```

```

INSERT INTO diningHall_mealplan (id, max_balance) VALUES (2, '1000.00');
INSERT INTO diningHall_mealplan (id, max_balance) VALUES (3, '10000.00');
-- Inserting Students
INSERT INTO diningHall_student (id, user_id, balance, meal_plan_id) VALUES (1, 3,
'0.00', 1);
INSERT INTO diningHall_student (id, user_id, balance, meal_plan_id) VALUES (2, 5,
'0.00', 2);
INSERT INTO diningHall_student (id, user_id, balance, meal_plan_id) VALUES (3, 6,
'0.00', 3);
-- Inserting Menu Items
INSERT INTO diningHall_menu (menu_id, item_id, info, cost) VALUES (0, 'Pepperoni
Pizza', 'Pepperoni and cheese and dough', '9.99');
INSERT INTO diningHall_menu (menu_id, item_id, info, cost) VALUES (1, 'Cheese
Burger', 'Cheese, beef patty, whole grain buns', '11.99');
-- Inserting Purchase History
INSERT INTO diningHall_purchasehistory (id, student_id, item_id, dop, cost) VALUES
(3, 1, 'Cheese Burger', '2025-02-14', '11.99');
INSERT INTO diningHall_purchasehistory (id, student_id, item_id, dop, cost) VALUES
(5, 3, 'Cheese Burger', '2025-04-02', '11.99');
INSERT INTO diningHall_purchasehistory (id, student_id, item_id, dop, cost) VALUES
(6, 1, 'Pepperoni Pizza', '2025-04-27', '11.99');
INSERT INTO diningHall_purchasehistory (id, student_id, item_id, dop, cost) VALUES
(7, 10, 'Pepperoni Pizza', '2025-04-27', '11.99');
-- Inserting Feedback
INSERT INTO diningHall_feedback (id, response) VALUES (1, 'Service: Service, Name:
Testing, Rating: 3, Desc: It's alright');
-- Inserting Report
INSERT INTO diningHall_report (item_id, amount) VALUES ('Cheese Burger', 2);
INSERT INTO diningHall_report (item_id, amount) VALUES ('Pepperoni Pizza', 1);

```

Feature Implementations

1. Student Registration

- a. **User Story:** As a student, I want to register a password so that I can be given a student number and log into the meal plan service. **AND** As a student, I want to select a meal plan so I can get access to funds to pay for meals.
- b. Students create their account using a student ID, password, and by selecting a meal plan. Student's cannot create an account using an already in use student ID, nor can they create one using an invalid ID. Passwords are encrypted and stored in the user/student table along with the student ID and meal plan choice.

2. Student Login

- a. **User Story:** As a student, I want to register a password so that I can be given a student number and log into the meal plan service.
- b. Students login using their student ID and created password. This is checked with the user/student table and either allows the student to login, or throws an error.

3. Student Menu View

- a. **User Story:** As a student, I want to be able to see each menu item and its nutritional info so I can plan what I want to eat.
- b. Once logged in, students can view all available menus and their relevant menu items. These are pulled from the menu table.

4. Student Balance and History View:

- a. **User Story:** As a student, I want to be able to see my past purchase history so that I can see how I've used my meal plan. **AND** As a student, I want to be able to access a meal plan tracker so that I can check my remaining balance.
- b. Once logged in, students can access their Balance/History page and view their remaining balance and entire purchase history. The balance is found by using the student ID in the student table and retrieving the balance. The

purchase history is grabbed by getting all purchase histories in the purchase history table that has the specified student ID.

5. Student Feedback Page:

- a. **User Story:** As a student, I want to be able to give ratings on food and service so any complaints can be properly addressed.
- b. Once logged in, students can give feedback on foods or services. Once all fields are filled out, the feedback is combined into a report and added to the feedback table for admin viewing.

6. Admin Dashboard:

- a. **User Story:** As an administrator, I want to be able to manage the menus, meal plans, and usage reports so that I can keep the meal plan service functioning properly. **AND** As an administrator, I want to be able to see feedback on services, so we can address any concerns or health and safety issues. **AND** As an administrator, I want to be able to track every item purchased so that I can manage usage reports.
- b. Admins can add, edit, or remove items from menus, open the feedback page to view all feedback, and look at the report to see what items are being bought the most. This dashboard uses various UI elements to allow the admin to view and alter the database itself for full control over the application.

Query Design

- Student.objects.select_related('meal_plan').filter(meal_plan__max_balance__gt=100)
 - SELECT `diningHall\_student`.`id`, `diningHall\_student`.`user\_id`,
`diningHall\_student`.`balance`, `diningHall\_student`.`meal\_plan\_id`,
`diningHall\_mealplan`.`id`, `diningHall\_mealplan`.`max\_balance` FROM
`diningHall\_student` INNER JOIN `diningHall\_mealplan` ON
(`diningHall\_student`.`meal\_plan\_id` = `diningHall\_mealplan`.`id`) WHERE
`diningHall\_mealplan`.`max\_balance` > 100
- PurchaseHistory.objects.values('item_id').annotate(num_purchases=Count('id'))
 - SELECT `diningHall\_purchasehistory`.`item\_id` AS `item\_id`,
COUNT(`diningHall\_purchasehistory`.`id`) AS `num\_purchases` FROM
`diningHall\_purchasehistory` GROUP BY 1 ORDER BY NULL
- Student.objects.filter(id__in=Subquery(PurchaseHistory.objects.filter(student=OuterRef('id')).values('student')).annotate(total_spent=Sum('cost')).filter(total_spent__gt=100).values('student'))
 - SELECT `diningHall\_student`.`id`, `diningHall\_student`.`user\_id`,
`diningHall\_student`.`balance`, `diningHall\_student`.`meal\_plan\_id` FROM
`diningHall\_student` WHERE `diningHall\_student`.`id` IN (SELECT
U0.`student\_id` AS `student` FROM `diningHall\_purchasehistory` U0 WHERE
U0.`student\_id` = (`diningHall\_student`.`id`) GROUP BY 1 HAVING
SUM(U0.`cost`) > 100 ORDER BY NULL)
- Student.objects.annotate(total_spent=Sum('purchasehistory__cost'))
 - SELECT `diningHall\_student`.`id`, `diningHall\_student`.`user\_id`,
`diningHall\_student`.`balance`, `diningHall\_student`.`meal\_plan\_id`,
SUM(`diningHall\_purchasehistory`.`cost`) AS `total\_spent` FROM
`diningHall\_student` LEFT OUTER JOIN `diningHall\_purchasehistory` ON
(`diningHall\_student`.`id` = `diningHall\_purchasehistory`.`student\_id`) GROUP
BY `diningHall\_student`.`id` ORDER BY NULL
- Student.objects.select_related('meal_plan').all()

- `SELECT `diningHall_student`.`id`, `diningHall_student`.`user_id`,
`diningHall_student`.`balance`, `diningHall_student`.`meal_plan_id`,
`diningHall_mealplan`.`id`, `diningHall_mealplan`.`max_balance` FROM
`diningHall_student` INNER JOIN `diningHall_mealplan` ON
(`diningHall_student`.`meal_plan_id` = `diningHall_mealplan`.`id`)`

User Interface Snapshots

Student Login Page (students log into their accounts. username/password checked with user database):

[Meal System](#) [Menus](#) [Balance/History](#) [Feedback](#)

Student ID:

Password:

Login

Create Account

Are you an administrator? [Click here](#)

Student Account Creation Page (students create their account. Checks for duplicates):

[Meal System](#) [Menus](#) [Balance/History](#) [Feedback](#)

Create an Account

Student ID: Password: Select Meal Plan:

Student Menu View (students view the available items (grabs all from menu table):

Meal System Menus Balance/History Feedback Welcome, 11811009! Logout			
Menu ID	Item Name	Nutritional Info	Cost
0	Pepperoni Pizza	Pepperoni and cheese and dough	9.99
1	Cheese Burger	Cheese, beef patty, whole grain buns	11.99

Student Balance and Purchase History View (students view their balance and history. Grabs student balance from student table, and only this student's purchase history from purchase history table):

Meal System Menus Balance/History Feedback Welcome, 11811009! Logout			
Balance: \$-11.01			
Username	Item Name	Date of Purchase	Cost
11811009	Pepperoni Pizza	04/02/2100	100.00
11811009	Cheeseburger	04/27/2025	11.99

Student Feedback Page (students can create feedback, adds to the feedback table):

Meal System

Menus

Balance/History

Feedback

Welcome, 11811009! Logout

Give Feedback

Select Feedback Type: Food

Name of Food or Service:

Select Rating: 1

Please Give a Brief Description:

Send

Admin Login Page (admins login here. Checks with user database):

Meal System

Menus

Balance/History

Feedback

Admin Username:

Password:

Login

Admin Dashboard (admins can edit menus, add an item to a student's purchase history, or access their other features):

Meal System

Django Dashboard

Welcome, admin! Logout

Menu Items

Feedback

Usage Reports

Manage Menu and Purchase History

Add Menu Item

Menu ID:

Item Name:

Nutritional Info:

Cost:

Add Menu Item

Add Purchase History

Username:

Item Name:

Date of Purchase:

Cost:

Add Purchase History

Update Menu Item

Select Menu Item: 0 - Pepperoni Pizza

Menu ID:

Item Name:

Nutritional Info:

Cost:

Save Changes

Delete Menu Item

Admin Menu View (admins can view the available menus):

Menu ID	Item Name	Nutritional Info	Cost
0	Pepperoni Pizza	Pepperoni and cheese and dough	9.99
1	Cheese Burger	Cheese, beef patty, whole grain buns	11.99

← Back to Dashboard

Admin Feedback View (admins can view all feedback):

Meal System Django Dashboard		Welcome, admin! Logout
ID	Response	
2	Service: Food, Name: Pepperoni Pizza, Rating: 2, Desc: Too Expensive! But pretty good!	
← Back to Dashboard		

Admin Usage Report View (admins can view the full usage report):

Meal System Django Dashboard		Welcome, admin! Logout
Item Name	Amount Purchased	
Cheese Burger	2	
Cheeseburger	2	
Pepperoni Pizza	1	
← Back to Dashboard		

Admin Database View (admins can edit all database info, including student data if they wish to change account info or unregister):

Django administration

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups

+ Add

Change

Users

+ Add

Change

DININGHALL

Feedback

+ Add

Change

Meal Plans

+ Add

Change

Purchase Histories

+ Add

Change

Reports

+ Add

Change

Students

+ Add

Change

Recent actions

My actions

Student object (11)

Student

PurchaseHistory object (2)

Purchase History

PurchaseHistory object (1)

Purchase History

Student object (10)

Student

MealPlan object (3)

Meal Plan

MealPlan object (2)

Meal Plan

MealPlan object (1)

Meal Plan

Limitations & Future Scope

There were few limitations that we ran into, besides our amount of time we spent on the project - I'm sure if we had more time or money to spend on the project, the project could be improved. Some future objectives could include:

- More robust login system - 2FA, passkeys, password resetting, etc.
- Add images to the menu items
- Add an official checkout system (either in-person with cashiers, or an ordering system)
- Less relying on the Django admin page for administration

Conclusion

In conclusion, while the project itself was engaging and provided valuable learning experiences, the process of preparing the accompanying reports and videos significantly added to the workload, which made the project sort of slug along. Despite these challenges, our group gained important knowledge and skills, making the experience worthwhile overall. We learned how to better use Django, Python, SQL, and HTML, and how to implement it in future projects. We also learned how to manage and structure projects to be efficient and productive.